

Conversa com o Gemini.

1. Prompt

Desenvolva este Trabalho de Conclusão de Curso seguindo uma abordagem incremental e iterativa baseada em metodologias ágeis. O projeto será dividido em sprints, com cronograma previsto até o dia 15 de outubro, permitindo entregas contínuas e evolução progressiva do sistema.

As sprints serão detalhadas em um documento separado, contendo a definição de requisitos, tarefas, estimativas e objetivos de cada ciclo. Essa divisão tem como finalidade organizar o desenvolvimento e facilitar o acompanhamento do progresso.

Além disso, haverá um segundo documento específico para o plano de prototipagem, no qual serão descritas as etapas de criação, validação e refinamento das interfaces e funcionalidades do sistema.

O desenvolvimento será realizado sprint por sprint, garantindo melhor controle, qualidade e adaptação ao longo do projeto. A cada ciclo, serão implementadas funcionalidades incrementais, permitindo validações frequentes e possíveis ajustes, contribuindo para um desenvolvimento mais eficiente e estruturado.

1.1. Resposta

É uma excelente iniciativa adotar metodologias ágeis e uma abordagem iterativa para o seu Trabalho de Conclusão de Curso (TCC). Essa organização em sprints e o planejamento detalhado de prototipagem e validação dão uma base muito sólida para o desenvolvimento do sistema de gestão de frotas hospitalares.

Apenas para garantir que estamos perfeitamente alinhados com a realidade do seu planejamento: você mencionou entregas até o dia 15 de outubro, mas, de acordo com o roadmap fornecido, a última sprint ("Polish + Documentação TCC") tem término previsto para 20 de outubro de 2026, com um *buffer* de segurança de 5 dias (21 a 25 de outubro). Trabalharemos com esse horizonte oficial em mente.

Como hoje é 27 de abril de 2026, nós já estamos oficialmente dentro do período da **Sprint 1**.

Abaixo, faço um breve resumo de como estamos estruturados para começar, baseado nos seus documentos.

Visão Geral Estratégica do Projeto

- **Carga Total Estimada:** 340 horas (200h para Requisitos Funcionais e 140h para Requisitos Não Funcionais) distribuídas em 13 sprints de 2 semanas.
- **Pilha de Tecnologias:**
 - **Mobile:** Flutter.
 - **Web:** React.js com Next.js.
 - **Backend & Dados:** Node.js com PostgreSQL.
 - **Nuvem:** Firebase (Authentication e Storage).
- **Qualidade e Validação:** O projeto prevê Prova de Conceito (PoC), testes automatizados (Jest e Cypress), testes de carga (K6/JMeter), avaliação heurística de Nielsen e validação prática com a persona do motorista.

Ponto de Partida: Fase 1 - Fundação

Estamos na sprint "**Setup + Autenticação**" (Período: 22/04 a 05/05), que possui uma carga de 28h.

Objetivos desta Sprint:

1. **RF:** Autenticar usuários (8h) - Motorista, central e recepcionista hospitalar.
2. **RNF:** Segurança e integridade dos dados (início, 20h).
3. **Observações de Implementação:** Configurar o repositório, o banco de dados PostgreSQL, o Firebase Auth e as variáveis de ambiente. Esta etapa define a base de todo o projeto.

Para iniciarmos nossa primeira iteração prática, por qual etapa da **Sprint 1** você gostaria de começar: configurando a modelagem inicial do banco de dados PostgreSQL, estruturando o boilerplate do backend em Node.js ou configurando o Firebase Auth?

2. Prompt

Muito boa pergunta, qual parte vc recomenda?

2.1. Resposta

Eu recomendo fortemente que você comece pela **Estruturação do Repositório e Modelagem do Banco de Dados (PostgreSQL)**.

No desenvolvimento de software, construir a aplicação "de baixo para cima" (da infraestrutura de dados para a lógica e, por fim, para as integrações externas) costuma ser o caminho mais seguro e lógico. Como o seu roadmap orienta a configuração do repositório, banco PostgreSQL, Firebase Auth e variáveis de ambiente, existe uma ordem natural para fazer essas peças se encaixarem.

Aqui está o fluxo de trabalho que considero mais eficiente para esta Sprint 1:

Fluxo de Trabalho Sugerido

1. **Repositório e Variáveis de Ambiente:** O primeiro passo prático é inicializar o Git e criar um arquivo `.env` para as variáveis de ambiente. Isso garante o versionamento desde a linha zero e evita que chaves sensíveis sejam expostas.
2. **Modelagem do Banco de Dados (PostgreSQL):** Antes de autenticar um usuário, precisamos saber onde e como os dados dele serão armazenados. Estruturar as tabelas iniciais para os perfis que o sistema exige (motorista, central e recepcionista hospitalar) é o alicerce do sistema.
3. **Boilerplate do Backend (Node.js):** Com o banco desenhado, inicializamos o projeto Node.js e configuramos a conexão do backend com o PostgreSQL.
4. **Integração com Firebase Auth:** Por fim, com a base pronta, configuramos o Firebase Authentication. O Firebase cuidará da segurança das senhas e da geração de tokens, e o seu backend em Node.js apenas fará a validação desses tokens e o cruzamento com o banco de dados.

Essa sequência atende perfeitamente ao requisito não funcional de iniciar o trabalho

focado em segurança e integridade, que possui uma carga de 20 horas prevista para esta etapa.

Para darmos o primeiro passo prático em direção ao código, você prefere que eu liste os comandos para a inicialização do repositório e do projeto Node.js, ou gostaria de focar primeiro em desenhar o script SQL para a criação das tabelas de usuários no PostgreSQL?

3. Prompt

Quero os passos completos justamente nesta ordem, como inicializo esse .env no git? Como acessar o PostgreSQL e fazer sua configuração, como configurar o boilerplate do backend, e por fim integração com o firebase

3.1. Resposta

Excelente escolha. Vamos construir a fundação do **Agiliza Frota** passo a passo.

Antes de começarmos, farei um pequeno ajuste de rota: nós **não** inicializamos o arquivo **.env** no Git. Pelo contrário, usamos o Git para ignorá-lo completamente. O arquivo **.env** guarda senhas, chaves secretas e configurações locais, então ele nunca deve ser enviado para o repositório online (como o GitHub), pois isso comprometeria a segurança do sistema.

Aqui está o guia completo para estruturar o backend em Node.js com arquitetura baseada em APIs REST.

1. Inicializando o Repositório e Protegendo o **.env**

Abra o seu terminal e execute os seguintes comandos para criar a pasta do projeto, inicializar o Git e o Node.js:

```
Bash
```

```
mkdir agilizafrota-backend
```

```
cd agilizafrota-backend
```

```
git init
```

```
npm init -y
```

Agora, crie os arquivos **.env** e **.gitignore** vazios:

```
Bash
```

```
touch .env .gitignore
```

Abra o arquivo **.gitignore** no seu editor de código e adicione o seguinte conteúdo

para proteger suas chaves e evitar o envio de arquivos pesados:

Plaintext

node_modules/

.env

firebase-key.json

2. Acesso e Configuração do PostgreSQL

O PostgreSQL foi escolhido pois oferece suporte a dados estruturados e geolocalização. Para o ambiente de desenvolvimento, você pode instalá-lo localmente (usando ferramentas como o pgAdmin) ou utilizar o Docker.

No seu arquivo **.env**, você configurará a string de conexão com o banco local:

Snippet de código

PORT=3000

DATABASE_URL=postgresql://seu_usuario:sua_senha@localhost:5432/agilizafrota

No seu banco de dados, você precisará de uma tabela inicial para atender ao requisito de autenticar os três tipos de usuários exigidos: motorista, central e recepcionista hospitalar. O script SQL inicial fica assim:

SQL

CREATE TABLE usuarios (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

nome VARCHAR(100) NOT NULL,

email VARCHAR(100) UNIQUE NOT NULL,

papel VARCHAR(20) CHECK (papel IN ('motorista', 'central', 'recepcionista'))
NOT NULL,

firebase_uid VARCHAR(128) UNIQUE NOT NULL,

criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);

*(Nota: a coluna **firebase_uid** é crucial, pois é ela que fará a ponte entre o sistema de login do Google e o seu banco de dados).*

3. Configurando o Boilerplate do Backend (Node.js)

Vamos instalar os pacotes essenciais: o framework web (**express**), o driver do Postgres (**pg**), o leitor de variáveis de ambiente (**dotenv**) e as permissões de segurança de domínio (**cors**).

Bash

```
npm install express pg dotenv cors
```

```
npm install --save-dev nodemon
```

Crie um arquivo chamado **server.js** na raiz do projeto. Este será o coração da sua API:

JavaScript

```
require('dotenv').config();
```

```
const express = require('express');
```

```
const cors = require('cors');
```

```
const { Pool } = require('pg');
```

```
const app = express();
```

```
app.use(cors());
```

```
app.use(express.json());
```

```
// Conexão com o PostgreSQL
```

```
const pool = new Pool({  
  connectionString: process.env.DATABASE_URL,  
});
```

```
app.get('/api/status', (req, res) => {  
  res.json({ mensagem: 'Agiliza Frota API rodando com sucesso!' });  
});
```

```
const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {

  console.log(`Servidor rodando na porta ${PORT}`);

});
```

*(Dica: no seu **package.json**, adicione um script **"dev": "nodemon server.js"** para facilitar a execução).*

4. Integração com o Firebase Authentication

O Firebase Authentication será responsável por autenticar os usuários de forma segura. O seu backend Node.js não vai gerenciar senhas; ele apenas vai validar se o token gerado pelo aplicativo (que o usuário usou para logar) é autêntico.

Primeiro, instale o SDK Admin do Firebase:

Bash

```
npm install firebase-admin
```

Em seguida, faça o seguinte no console online do Firebase:

1. Crie seu projeto e ative o Authentication.
2. Vá na engrenagem superior esquerda > **Configurações do Projeto** > **Contas de Serviço**.
3. Clique em **Gerar nova chave privada**.
4. Salve o arquivo **.json** baixado na raiz do seu projeto com o nome **firebase-key.json** (lembra que já colocamos ele no **.gitignore** no passo 1?).

Crie um arquivo chamado **firebase.js** na raiz do projeto para inicializar a conexão com a nuvem do Google:

JavaScript

```
const admin = require('firebase-admin');

const serviceAccount = require('./firebase-key.json');

admin.initializeApp({

  credential: admin.credential.cert(serviceAccount)

});
```

```
module.exports = admin;
```

Com isso, a fundação técnica da Sprint 1 está arquitetada. O repositório está seguro, a modelagem do banco suporta os papéis necessários, a API está de pé e o Firebase está integrado.

Para validarmos essa etapa e começarmos a programar as rotas, você prefere focar em configurar esse banco PostgreSQL localmente agora, ou deseja que eu escreva o código do *middleware* (a função que vai interceptar as requisições, ler o token do Firebase e permitir a entrada no sistema)?

4. Prompt

Quero começar pelo o que vc indicar

4.1. Resposta

Como hoje já é dia 5 de maio de 2026, nós estamos exatamente no último dia da Sprint de "Setup + Autenticação"! Para garantirmos a entrega do seu Requisito Funcional (RF) de "Autenticar usuários" dentro do prazo estipulado, minha recomendação de ouro é irmos direto para a **criação do Middleware de Autenticação**.

Por que essa escolha? Você já tem o banco PostgreSQL para salvar os dados e o Firebase inicializado no backend. O middleware é a "porta de segurança" que vai interceptar as requisições do aplicativo mobile (Flutter), ler o token do Firebase e permitir ou negar a entrada no sistema. É ele que transforma sua API em uma API segura.

Vamos colocar a mão no código!

1. Criando o Middleware

Na raiz do seu projeto `agilizafrota-backend`, crie uma pasta chamada `middlewares` e, dentro dela, um arquivo chamado `authMiddleware.js`.

JavaScript

```
const admin = require('../firebase'); // Importa a configuração do Firebase que fizemos antes
```

```
const verificarToken = async (req, res, next) => {
```

```
  // 1. Pega o token do cabeçalho da requisição
```

```
  const tokenHeader = req.headers.authorization;
```

```
if (!tokenHeader || !tokenHeader.startsWith('Bearer ')) {  
    return res.status(401).json({ erro: 'Acesso negado. Token não fornecido ou  
formato inválido.' });  
}  
  
const token = tokenHeader.split(' ')[1];  
  
try {  
    // 2. Pede para o Firebase validar se esse token é real e não expirou  
    const tokenDecodificado = await admin.auth().verifyIdToken(token);  
  
    // 3. Se deu certo, injetamos os dados do usuário (como o UID) na requisição  
    req.usuario = tokenDecodificado;  
  
    // 4. Libera a requisição para seguir o fluxo  
    next();  
} catch (erro) {  
    console.error('Erro na verificação do token:', erro);  
    return res.status(403).json({ erro: 'Token inválido ou expirado.' });  
}  
};  
  
module.exports = verificarToken;
```

2. Protegendo uma Rota na Prática

Agora, vamos voltar ao seu `server.js` e aplicar essa fechadura em uma rota nova para testar. Atualize o arquivo para ficar assim:

JavaScript

```
require('dotenv').config();  
  
const express = require('express');
```



```
const cors = require('cors');

const { Pool } = require('pg');

const verificarToken = require('./middlewares/authMiddleware'); // <-- Importando o
middleware

const app = express();

app.use(cors());

app.use(express.json());

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
});

// Rota pública (qualquer um acessa)
app.get('/api/status', (req, res) => {
  res.json({ mensagem: 'Agiliza Frota API rodando com sucesso!' });
});

// Rota PRIVADA (só acessa com token válido do Firebase)
app.get('/api/perfil', verificarToken, (req, res) => {
  // Como o middleware liberou, podemos acessar req.usuario
  res.json({
    mensagem: 'Acesso autorizado!',
    uid_do_firebase: req.usuario.uid,
    email: req.usuario.email
  });
});

const PORT = process.env.PORT || 3000;
```

```
app.listen(PORT, () => {  
  
  console.log(`Servidor rodando na porta ${PORT}`);  
  
});
```

Pronto! Com esse código, você fechou o ciclo do backend para a autenticação, garantindo a segurança e integridade dos dados, que é o Requisito Não Funcional (RNF) mais importante dessa primeira fase.

Como essa sprint se encerra oficialmente hoje, você quer que eu te mostre como testar essa rota protegida no Postman/Insomnia para validarmos a entrega, ou prefere que a gente já comece a planejar a arquitetura da "Sprint 2 - Cadastros e UX base" (que foca em gerenciar veículos, motoristas e unidades)?

5. Prompt

Agora quero focar na estruturação do banco de dados que vc mencionou anteriormente quando perguntou o que eu queria fazer.

5.1. Resposta

Perfeito. Como estamos na transição da **Sprint 1 (Fundação)** para a **Sprint 2 (Cadastros)**, estruturar o banco de dados agora é o passo mais estratégico. O PostgreSQL é excelente para o **Agiliza Frota** pela sua robustez e suporte a dados geográficos (importante para o rastreamento via GPS que virá depois).

Abaixo, apresento a modelagem lógica e o script SQL para as tabelas fundamentais: **Usuários, Unidades Hospitalares e Veículos**.

1. Modelagem Lógica (Entidades e Relacionamentos)

Para suportar as regras de negócio do seu TCC, precisamos que as tabelas se converse:

- **Unidades:** Representam os hospitais/pontos de apoio.
- **Usuários:** Ligados a um papel (role) e, no caso do recepcionista, a uma unidade específica.
- **Veículos:** Ligados a uma unidade de origem.

2. Script SQL de Estruturação (DDL)

Você pode executar este script no seu terminal do PostgreSQL ou via interface (como pgAdmin ou DBeaver):

SQL

-- Habilitar extensão para geração de UUIDs (IDs únicos e seguros)

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
```

-- 1. TABELA DE UNIDADES HOSPITALARES

```
CREATE TABLE unidades (
```

```
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
    nome VARCHAR(100) NOT NULL,
```

```
    endereco TEXT NOT NULL,
```

```
    cidade VARCHAR(50) DEFAULT 'Guarapuava', -- Exemplo baseado na sua  
localização
```

```
    lat DECIMAL(9,6), -- Para futuro rastreamento GPS
```

```
    lng DECIMAL(9,6),
```

```
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);
```

-- 2. TABELA DE USUÁRIOS (Integrada ao Firebase Auth)

```
CREATE TABLE usuarios (
```

```
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
    firebase_uid VARCHAR(128) UNIQUE NOT NULL, -- UID vindo do Firebase
```

```
    nome VARCHAR(100) NOT NULL,
```

```
    email VARCHAR(100) UNIQUE NOT NULL,
```

```
    papel VARCHAR(20) CHECK (papel IN ('motorista', 'central', 'recepcionista'))  
NOT NULL,
```

```
    unidade_id UUID REFERENCES unidades(id), -- Opcional: apenas para  
recepcionistas
```

```
    telefone VARCHAR(20),
```

```
    ativo BOOLEAN DEFAULT TRUE,
```

```
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);
```

-- 3. TABELA DE VEÍCULOS

```
CREATE TABLE veiculos (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    placa VARCHAR(10) UNIQUE NOT NULL,  
    modelo VARCHAR(50) NOT NULL,  
    marca VARCHAR(30),  
    ano INTEGER,  
    quilometragem_atual INTEGER DEFAULT 0,  
    status VARCHAR(20) CHECK (status IN ('disponivel', 'em_uso', 'manutencao'))  
    DEFAULT 'disponivel',  
    unidade_id UUID REFERENCES unidades(id),  
    ultima_revisao DATE,  
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

3. Por que estruturamos assim?

1. **UUID vs Serial:** Usamos **UUID** em vez de números sequenciais (1, 2, 3...). Isso aumenta a segurança (impede que alguém descubra IDs por tentativa e erro) e facilita a sincronização com o modo offline que você previu na Fase 5.
2. **Firebase_uid:** Esta coluna é a sua "âncora". Quando o usuário logar no app (Flutter), o Firebase retornará um UID. Sua API usará esse UID para buscar no banco qual é o **papel** (motorista ou central) e as permissões dele.
3. **Check Constraints:** Note os comandos **CHECK (papel IN (...))**. Isso garante integridade total: o banco de dados impedirá que alguém cadastre um "administrador" se o sistema só aceita "motorista", "central" ou "recepcionista".

4. Próximo Passo Sugerido

Com as tabelas criadas, o próximo passo lógico dentro da **Sprint 2** (que começa amanhã, 6 de maio) é o **CRUD de Veículos e Motoristas**.

Você prefere que eu ajude a criar as **Rotas no Node.js** para cadastrar um novo veículo ou quer focar na criação de uma **Unidade Hospitalar** de teste para popular o banco?